

Graph Theory Research Paper Generator

Phase-wise Implementation Plan

6 Phases structured stages	36 Steps granular tasks
18 Days estimated timeline	Free Tier Gemini + open source

Stack: LangGraph · Gemini 3.1 Flash Lite · ChromaDB · ArXiv · Semantic Scholar · Streamlit · Pandoc

Phase overview

Phase	Title	Timeline	Deliverable
Phase 1	Project scaffold & state schema	Day 1–2	Runnable LangGraph skeleton with shared state
Phase 2	Retrieval pipeline	Day 3–5	RAG store populated with real graph theory papers
Phase 3	Human checkpoint 1 + gap finder	Day 6–8	Human-approved paper set + ranked research gaps
Phase 4	Writer + critic loop	Day 9–12	Self-correcting grounded draft with inline citations
Phase 5	Evaluation layer	Day 13–15	Automated eval scores attached to every paper run
Phase 6	Human checkpoint 3 + PDF output	Day 16–18	Validated PDF research paper with full citation list

Tag legend

LangGraph	Gemini	Eval	RAG	Streamlit
-----------	--------	------	-----	-----------

Phase 1
Day 1–2**Project scaffold & state schema***Deliverable: Runnable LangGraph skeleton with shared state*

#	Step	Description & tags
1.1	Set up project structure	Create folders: agents/, eval/, retrieval/, ui/, utils/. Init a Python virtual env, pin versions in requirements.txt.
1.2	Install core dependencies	pip install langgraph langchain-google-genai chromadb sentence-transformers arxiv semanticsscholar streamlit [LangGraph] [Gemini]
1.3	Configure Gemini API key	Store in .env via python-dotenv. Init google-generativeai client with gemini-3.1-flash-lite as default model. [Gemini]
1.4	Define AgentState TypedDict	Fields: research_question, sub_queries, retrieved_papers, embeddings_ready, gaps, selected_gap, outline, draft_sections, critic_feedback, revision_count, citation_report, human_decisions, eval_scores. [LangGraph]
1.5	Build empty LangGraph graph	Create StateGraph(AgentState). Register all 6 node stubs that just pass state through. Compile and verify graph.invoke() runs end-to-end without errors. [LangGraph]
1.6	Add SqliteSaver checkpointer	Attach SqliteSaver to the compiled graph so every node's output is persisted to a local .db file. Test that resuming from a checkpoint works. [LangGraph]

Phase 2

Day 3–5

Retrieval pipeline*Deliverable: RAG store populated with real graph theory papers*

#	Step	Description & tags
2.1	Planner agent — query decomposition	Prompt Gemini to decompose the research question into 6–8 sub-queries specific to graph theory subfields. Parse response into a list and write to state.sub_queries. [Gemini]
2.2	ArXiv fetcher	Use the arxiv Python library. For each sub-query, fetch top 10 papers filtered to cs.DM, math.CO categories. Store title, abstract, authors, DOI, PDF URL. [RAG]
2.3	Semantic Scholar fetcher	Call the free S2 API (/graph/v1/paper/search). Fetch fieldsOfStudy=Mathematics for each sub-query. De-duplicate against ArXiv results by DOI. [RAG]
2.4	Embed and index into ChromaDB	Use Google's text-embedding-004 (free) to embed each abstract. Store in a local ChromaDB collection with metadata: source, doi, year, authors. [RAG] [Gemini]
2.5	Retriever node — similarity search	For each sub-query, run a top-k=5 similarity search against ChromaDB. Merge results, re-rank by relevance score, write deduplicated list to state.retrieved_papers. [RAG] [LangGraph]
2.6	Test retrieval end-to-end	Run with a known graph theory question (e.g. chromatic polynomials). Manually inspect retrieved papers for relevance. Tune top-k and similarity threshold. [RAG]

Phase 3

Day 6–8

Human checkpoint 1 + gap finder*Deliverable: Human-approved paper set + ranked research gaps*

#	Step	Description & tags
3.1	Wire interrupt_before on retriever node	Add interrupt_before=['retriever'] to graph.compile(). The graph pauses before the node executes, exposing state to the UI. [LangGraph]
3.2	Build Streamlit checkpoint 1 UI	Display retrieved papers as cards (title, abstract snippet, source badge). Add approve/remove toggles per paper and a re-query text field. Write human decisions to state.human_decisions. [Streamlit]
3.3	Resume graph after human input	On form submit, call graph.invoke(Command(resume=human_decisions), config=thread_config) to resume from the checkpoint with updated state. [LangGraph] [Streamlit]
3.4	Gap finder agent — literature mapping	Send all approved abstracts in a single Gemini call (use the 1M context window). Prompt: identify open conjectures, unexplored graph families, missing algorithmic proofs, unconnected subfields. [Gemini]
3.5	Structure gap output	Prompt Gemini to return gaps as JSON: [{gap_title, description, supporting_evidence: [doi], novelty_score}]. Parse and write to state.gaps. [Gemini]
3.6	Human checkpoint 2 — gap selection	Show gaps in Streamlit as cards with novelty scores. Human selects one gap and optionally edits its framing. Write to state.selected_gap. [Streamlit] [LangGraph]

Phase 4
Day 9–12

Writer + critic loop
Deliverable: Self-correcting grounded draft with inline citations

#	Step	Description & tags
4.1	Writer agent — section-by-section	For each section (abstract, intro, related work, gap analysis, proposed approach, conclusion), retrieve top-5 relevant chunks from ChromaDB and inject as context. Every sentence must include a [SOURCE_ID] tag. [Gemini] [RAG]
4.2	Enforce grounding in writer prompt	System prompt: "You may only write claims directly supported by provided source passages. End every sentence with [SOURCE_ID]. If you cannot cite a claim, omit it." [Gemini]
4.3	Critic agent — grounding check	For each sentence, extract the [SOURCE_ID] and verify the claim is entailed by that source's abstract. Flag sentences where the source does not support the claim. Score 0–10. [Gemini] [Eval]
4.4	Critic agent — coherence check	Evaluate logical flow between sections, whether the gap is addressed, and whether the abstract matches the body. Return structured feedback: {score, flagged_sentences, missing_sections}. [Gemini] [Eval]
4.5	Conditional edge — pass or revise	Add a route_after_critic(state) function. If critic score >= 7 and flagged_sentences < 3, route to checkpoint 3. Else route back to writer with critic feedback injected. Cap at 3 revision cycles. [LangGraph]
4.6	Track revision history in state	Append each (draft, critic_feedback, score) tuple to state. This becomes your training data for future prompt improvements. [LangGraph]

Phase 5

Day 13–15

Evaluation layer*Deliverable: Automated eval scores attached to every paper run*

#	Step	Description & tags
5.1	NLI grounding evaluator	Install cross-encoder/nli-deberta-v3-small via HuggingFace. For every (sentence, cited_abstract) pair, run NLI inference. Label: entailment=grounded, neutral=weak, contradiction=hallucination. [Eval]
5.2	BERTScore evaluator	pip install bert-score. Compute F1 between each generated section and its pool of cited sources. Log per-section scores to state.eval_scores. [Eval]
5.3	Citation DOI verifier	For every [SOURCE_ID] in the draft, resolve the DOI via CrossRef API (free, no key needed). Flag any DOI that returns 404 as a hallucinated citation. [Eval]
5.4	LLM-as-judge rubric	Write a Gemini eval prompt with a 5-axis rubric: grounding (0–3), novelty of gap (0–3), section completeness (0–3), logical coherence (0–3), academic tone (0–3). Return JSON scores + reasoning. [Gemini] [Eval]
5.5	Structural completeness check	Rule-based: assert paper has all 6 sections, each section >= min_word_count (abstract:150, intro:300, related:400, gap:300, approach:400, conclusion:200). Log pass/fail per section. [Eval]
5.6	Aggregate eval dashboard	Build a Streamlit eval tab. Display per-run scores, NLI distribution pie chart, BERTScore per section bar chart, and flagged sentences list. Store all scores to a local SQLite evals table. [Streamlit] [Eval]

Phase 6
Day 16–18

Human checkpoint 3 + PDF output
Deliverable: Validated PDF research paper with full citation list

#	Step	Description & tags
6.1	Human checkpoint 3 — final review UI	Show the full draft in Streamlit with eval scores alongside each section. Human can inline-edit any section, approve, or send specific sections back for revision with notes. [Streamlit] [LangGraph]
6.2	Citation formatter	Resolve all [SOURCE_ID] references to full metadata via Semantic Scholar API. Format as IEEE or APA using a Gemini formatting prompt. Build a deduplicated references list. [Gemini]
6.3	Markdown assembly	Assemble all approved sections + references into a single well-structured .md file. Replace [SOURCE_ID] inline tags with numbered superscripts.
6.4	Pandoc → LaTeX → PDF	Run: pandoc paper.md -o paper.pdf --pdf-engine=xelatex --csl=ieee.csl --bibliography=refs.bib. Install texlive-xetex for the LaTeX engine. Verify output renders correctly.
6.5	LangSmith tracing	Set LANGCHAIN_TRACING_V2=true and LANGCHAIN_API_KEY in .env. All LangGraph node runs auto-log. Use the LangSmith dashboard to inspect per-node latency, token counts, and errors. [LangGraph]
6.6	Rate limit handling	Add exponential backoff (tenacity library) around all Gemini calls. Queue sub-queries in batches of 3 with 4s delay between batches to stay within 15 RPM free tier limits. [Gemini]